

벡터 스토어



1. 벡터 스토어 개념



LangChain에서 **Vector Store**(벡터 저장소) 는 텍스트, 문서 등의 임베딩 벡터를 저장하고 검색하는 역할을 한다.

아래는 주요 **Vector Store** 종류(백엔드별 분류) 와 각 특징이다 📌



1. In-Memory / Local Type

이름	설명	특징
FAISS	Facebook AI가 만든 고속 벡터 검색 라이브러리	CPU/GPU 지원, 빠른 최근접 탐색, 로컬 개발용으로 가장 널리 사용
Chroma	LangChain 기본 내장형 벡터 DB	설치 간단, 메타데이터 관리 용이, Python 친화적
DocArray	DeepMind 개발의 문서·벡터 통합 스토리지	문서+벡터 통합 관리, JSON 기반 저장
SKLearnVectorStore	scikit-learn의 NearestNeighbors 기반	단순 벡터 비교 실습용
MemoryVectorStore	휘발성 메모리 기반	테스트, 데모용, 세션 종료 시 데이터 소멸

2. Database / Cloud Type

이름	설명	특징
PostgreSQL + pgvector	확장 모듈(pgvector)로 벡터 검색 지원	SQL + 벡터 검색 결합, 운영 서비스에 적합
Pinecone	상용 클라우드형 벡터 DB	빠른 인덱싱, API 중심, 대규모 데이터용
Weaviate	오픈소스/클라우드형 벡터 DB	GraphQL 기반, 자동 스키마, 클라우드 버전 존재
Milvus	오픈소스 분산 벡터 DB	대용량, GPU 가속, 기업용 배포에 적합
Qdrant	Rust 기반 오픈소스	빠르고 경량, Docker로 쉽게 배포 가능
RedisVectorStore	Redis 7.0+의 벡터 검색 기능 사용	캐싱 + 벡터 검색 통합, 실시간 서비스에 유리
ElasticsearchVectorSearch	Elasticsearch 8.x 이후 지원	텍스트 검색 + 벡터 검색 결합, 기존 ES 사용자에게 적합
SupabaseVectorStore	Supabase(PostgreSQL 기반) 확장	무료 요금제 존재, REST API 지원
Azure Cognitive Search	Azure에서 제공하는 하이브리드 검색	OpenAI + Azure 통합 시 유용
Google Vertex AI Matching Engine	Google Cloud의 벡터 검색 서비스	Vertex AI 통합, 대규모 AI 서비스용

3. 특수 목적형 / 기타

이름	설명	특징
Alibaba OpenSearch	Alibaba Cloud의 벡터 검색 엔진	중국 내 서비스 중심
AnalyticDB VectorStore	Alibaba AnalyticDB 기반	SQL + 벡터 혼합 분석 가능
TiDB Vector Store	PingCAP의 HTAP DB에 벡터 기능 추가	실시간 분석 + 검색용
DuckDBVectorStore	DuckDB를 벡터 DB처럼 사용	로컬 분석, 실습용
Zilliz Cloud	Milvus 클라우드 버전	관리형 서비스, 설정 단순

4. LangChain 연동 코드 예시

python

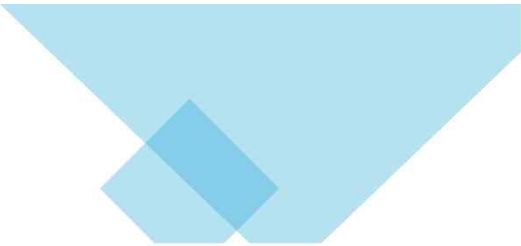
```
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings

# 1) 임베딩 모델 정의
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# 2) 벡터 스토어 생성
texts = ["AI is the future.", "LangChain helps build AI apps.", "Vector DB stores embeddings."]
vectorstore = FAISS.from_texts(texts, embedding=embeddings)

# 3) 검색 예시
query = "What helps in building AI applications?"
docs = vectorstore.similarity_search(query, k=2)

for doc in docs:
    print(doc.page_content)
```



요약

구분	대표 예시	특징
로컬형(Local)	FAISS, Chroma	빠르고 설정 간단, 실습/소규모 프로젝트
클라우드형(Managed)	Pinecone, Weaviate, Milvus Cloud, Qdrant Cloud	대규모 데이터, API 기반 운영 서비스
SQL 통합형	PostgreSQL(pgvector), Supabase	기존 DB와 통합, 관리 편의성
하이브리드 검색형	Elasticsearch, Azure Search	텍스트+벡터 동시 검색 가능



💰 LangChain 주요 Vector Store 요금 비교표

구분	이름	무료 사용 여부	무료 사용 범위	유료 전환 조건 / 요금 구조	비고
🟡 로컬형(Local)	FAISS	✅ 완전 무료	로컬에서 무제한 사용 가능	없음	Facebook 오픈소스, pip 설치만으로 가능
	Chroma	✅ 무료	로컬 실행시 무제한	Chroma Cloud(향후 유료 예정)	LangChain 기본 연동 추천
	DocArray	✅ 무료	로컬 저장 무제한	클라우드 사용 시 유료 (Jina AI 계정 필요)	문서+벡터 혼합 저장
	SKLearnVectorStore	✅ 무료	로컬 사용 제한 없음	없음	교육용·테스트용
	MemoryVectorStore	✅ 무료	세션 내 임시 메모리	세션 종료 시 데이터 삭제	데모용

구분	이름	무료 사용 여부	무료 사용 범위	유료 전환 조건 / 요금 구조	비고
클라우드형 (Managed)	Pinecone	⚡ 일부 무료	Starter Tier (최대 1GB)	초과 시 유료 (시간·용량 기반 과금)	상용 서비스 중심
	Weaviate Cloud / 오픈소스 버전	✅ 무료 (오픈소스)	로컬 / Docker 배포 가능	SaaS Cloud 요금제 존재	GraphQL 기반 검색
	Milvus	✅ 무료 (오픈소스)	Docker 로컬 실행	Zilliz Cloud(상용) 사용 시 유료	대용량 데이터 지원
	Qdrant	✅ 무료 (오픈소스)	로컬 / Docker 무제한	Qdrant Cloud(유료 플랜)	Rust 기반, 경량·빠름
	RedisVectorStore	⚡ 제한적 무료	Redis Stack 로컬 사용 무료	Redis Cloud 사용 시 유료	메모리 중심, 실시간
	ElasticsearchVectorSearch	⚡ 제한적 무료	로컬 ES 오픈소스 버전 사용 가능	Elastic Cloud 유료 플랜	텍스트+벡터 통합 검색
	SupabaseVectorStore	✅ 무료	Free Plan (500MB DB)	DB 공간 초과 시 유료	PostgreSQL + pgvector 기반
	Azure Cognitive Search	❌ 유료	없음	사용량 기반 요금 (Azure Search)	OpenAI + Azure 통합형
	Google Vertex AI Matching Engine	❌ 유료	없음	요청 수·저장량 기준 과금	대기업·클라우드 서비스용



정리 요약

범주	무료 사용 가능 여부	설명
 완전 무료형	FAISS, Chroma(Local), DocArray(Local), SKLearnVectorStore, MemoryVectorStore	로컬 환경에서 완전 무료
 부분 무료형 (Free Tier 제공)	Pinecone, Qdrant Cloud, Supabase, Redis Stack, Weaviate Cloud	초기 테스트용 무료 플랜 제공
 유료형 (클라우드 서비스)	Azure Cognitive Search, Google Vertex AI Matching Engine	대기업용 / 상용 운영 목적
 추천 조합 (무료로 시작)	LangChain + OpenAI Embedding + FAISS(또는 Chroma)	설치 간단, 100% 무료



2. FAISS (Facebook AI Similarity Search)

FAISS는 **LangChain**에서 가장 널리 사용되는 로컬 벡터스토어(Vector Store) 로, 빠른 벡터 유사도 검색을 위해 만들어진 **Facebook AI Research**의 오픈소스 라이브러리

1. FAISS의 핵심 개념

FAISS는 "유사도 검색(Similarity Search)"을 위해 만들어졌다.

즉, 임베딩(embedding)된 문장이나 문서 벡터 중에서 쿼리 벡터와 가장 가까운(유사한) 항목을 효율적으로 찾아준다.

기본 원리

1. 텍스트 → 벡터 변환 (Embedding)

OpenAI의 `text-embedding-3-small` 같은 모델을 사용해 문장을 수치 벡터로 변환한다.

예) "서울의 날씨" → `[0.12, -0.98, 0.45, ...]`

2. 벡터 저장 (FAISS Index)

변환된 벡터를 FAISS 인덱스에 저장한다.

3. 검색 시 벡터 유사도 비교

질문(쿼리)을 임베딩 후, 기존 벡터들과 코사인 유사도(Cosine Similarity) 또는 L2 거리(Euclidean Distance) 기준으로 비교해 가장 가까운 벡터를 반환한다.

2. 장점

항목	설명
빠른 검색 속도	수백만 개의 벡터 중에서도 실시간 검색 가능 (GPU 가속 지원)
대규모 데이터 처리	수십 GB 이상 임베딩 데이터도 처리 가능
여러 인덱스 구조 지원	Flat, IVF, PQ, HNSW 등 다양한 검색 최적화 알고리즘
무료 오픈소스	Apache License 2.0, 상업용 사용 가능

3. 단점

항목	설명
데이터 영속성 부족	메모리 기반이라, DB처럼 영구 저장하려면 별도 파일 저장 필요
분산 환경 미지원	여러 서버에 나눠 저장하는 분산 인덱스는 직접 구현해야 함
SQL 쿼리 불가	벡터 검색만 가능, 필터링이나 조인 기능 없음
운영 관리 어려움	RDB처럼 자동 백업/복구 기능이 없음

4. FAISS 인덱스 종류

인덱스	특징	사용 예시
IndexFlatL2	정확도 100%, 거리 계산 단순	소규모 데이터, 학습 없이 바로 사용
IVF (Inverted File)	벡터를 클러스터링해 검색속도 향상	중간 규모 데이터
PQ (Product Quantization)	벡터를 압축해 메모리 절감	대규모 데이터
HNSW	그래프 기반 근사 최근접 탐색	매우 빠른 검색, 정확도 높음

5. LangChain에서 FAISS 사용하는 예제

python

```
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings

# 1. 임베딩 모델 정의
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# 2. 문서와 벡터 저장
texts = ["서울의 날씨는 맑음", "부산의 해운대는 아름답다", "제주는 바람이 세다"]
faiss_store = FAISS.from_texts(texts, embedding=embeddings)

# 3. 검색 (쿼리와 유사한 문장 찾기)
query = "날씨가 좋은 도시"
results = faiss_store.similarity_search(query, k=2)

for r in results:
    print(r.page_content)
```

6. 다른 벡터 스토어와의 비교

구분	FAISS	Chroma	PostgreSQL + pgvector
저장 방식	메모리 기반	로컬 DB	RDB 확장
속도	매우 빠름	빠름	중간
데이터 영속성	파일로 직접 저장 필요	자동 영속	DB 자동 저장
분산 지원	X	X	O (DB 수준)
무료 여부	O	O	O
적합한 용도	빠른 임시 검색, 실험	로컬 RAG 서비스	기업용, 지속 저장형

7. 정리

- **FAISS** = “벡터 검색을 위한 GPU 가속 라이브러리”
- RAG 시스템에서 문서 임베딩 → 저장 → 유사도 검색 단계의 핵심 역할을 한다.
- 학습용이나 데모, 빠른 검색 중심의 프로젝트에 적합하다.
- 반면, 데이터 영속성과 필터링이 필요한 환경에서는 **pgvector** 같은 DB형 벡터 스토어가 더 유용하다.

FAISS가 벡터 간 거리 계산(유사도 계산) 과정을 GPU(Graphics Processing Unit) 로 처리할 수 있다

FAISS (또는 다른 벡터 저장소)에서 자주 쓰이는 `from_texts()`와 `from_documents()`는 LangChain이 벡터 인덱스를 "처음 생성할 때" 사용하는 클래스 메서드다. 두 메서드는 비슷해 보이지만, 입력 데이터의 형태와 메타데이터 관리 방식이 다르다.

● 개요

구분	<code>from_texts()</code>	<code>from_documents()</code>
입력 형태	단순 문자열 리스트 (<code>list[str]</code>)	<code>Document</code> 객체 리스트 (<code>list[Document]</code>)
메타데이터 지원	지원하지 않음	지원함 (<code>metadata</code> 필드 포함)
적합한 용도	간단한 실험, 테스트용	PDF·웹문서 등 실제 데이터 기반 RAG
반환 결과	문장 내용만 포함된 <code>Document</code> 객체	내용 + 메타데이터가 포함된 <code>Document</code> 객체

● FAISS 인덱스 관리 메서드 비교표

메서드	역할	주요 인자	반환값	사용 시점	특징 / 주의사항
<code>save_local(path)</code>	현재 메모리에 있는 인덱스를 로컬 폴더에 저장	<code>path</code> : 저장 경로 (예: <code>"faiss_index_store"</code>)	없음	인덱스를 새로 만들거나 업데이트한 후	• <code>index.faiss</code>, <code>index.pkl</code> 두 파일로 저장 • 재시작 후에도 같은 데이터 재사용 가능
<code>load_local(path, embedding, allow_dangerous_deserialization=True)</code>	저장된 인덱스를 복원(불러오기)	<code>path</code> : 저장 폴더 <code>embedding</code> : 동일한 임베딩 모델 객체 <code>allow_dangerous_deserialization</code> : pickle 허용 여부	<code>FAISS</code> 인스턴스	기존 인덱스를 다시 사용하고 싶을 때	• 저장 시 사용한 임베딩 모델과 동일해야 함 • pickle 역직렬화 경고 시 True 설정 필요
<code>merge_from(other_store)</code>	다른 FAISS 인덱스와 병합 (merge)	<code>other_store</code> : 다른 FAISS 객체	없음	여러 인덱스를 하나로 합칠 때	• 두 인덱스의 임베딩 차원이 같아야 함 • 병합 후 <code>save_local()</code> 로 다시 저장 가능

3. Chroma



● Chroma란?

Chroma는 오픈소스 벡터 데이터베이스(Vector Database) 로,
문장·문서·이미지 등에서 임베딩(Embedding)된 벡터를 **영구적으로 저장하고**,
유사도 검색(Similarity Search) 을 빠르게 수행할 수 있는 시스템이다.

한마디로 말하면,

“FAISS가 빠른 검색용 인덱스라면, Chroma는 데이터베이스형 벡터 저장소” 이다.

● 주요 특징

항목	설명
엔진 타입	내장형(Local) 데이터베이스, 디스크 기반 영구 저장
지원 기능	벡터 저장, 삭제, 업데이트, 메타데이터 필터링, 유사도 검색
호환성	LangChain, LlamaIndex, OpenAI, Hugging Face 등과 호환
언어 지원	Python, JavaScript 등
라이선스	오픈소스 (Apache 2.0)
서버모드 지원	Chroma Cloud / Server 모드도 지원 (HTTP API 가능)

● 발표 배경

Chroma는 2022~2023년 AI 생태계에서 **벡터 데이터베이스(Vector DB)** 수요가 폭발하면서 등장했습니다. 당시 **FAISS**(Meta, 2017), **Milvus**(Zilliz), **Weaviate**, **Pinecone** 등이 있었는데, 이들은 대부분 서버형/클라우드형 구조로 배포가 복잡했습니다.

이에 비해 **Chroma는 "개발자가 로컬에서 바로 쓸 수 있는 가볍고 단순한 벡터DB"**를 목표로 만들어졌습니다.

● 주요 발표 내용 (2023년 공개 당시)

- "Embeddings를 로컬에서 쉽게 저장하고 관리하자"
- Python 개발자 친화적인 API (`from_texts()`, `add_texts()`, `similarity_search()`)
- 데이터 영구 저장 (`persist_directory` 기반 SQLite + Parquet 구조)
- **LangChain, LlamaIndex, OpenAI 등과 완전 통합 지원**

공식 문구 중 하나는

"Chroma is the open-source embedding database built for developers."

(개발자를 위한 오픈소스 임베딩 데이터베이스)

● FAISS vs Chroma 차이

비교 항목	FAISS	Chroma
역할	인덱스 기반 검색 엔진	데이터베이스형 벡터 저장소
저장 방식	메모리 기반 (파일 저장 가능)	로컬 DB (디스크 자동 저장)
데이터 추가/삭제	추가/삭제 시 수동 재인덱싱 필요	<code>.add_texts()</code> , <code>.delete()</code> 로 동적 관리
메타데이터 필터링	제한적	강력 (조건 필터 지원)
분산/클라우드 운영	직접 구현 필요	Chroma Cloud / Docker 기반 지원
속도	매우 빠름 (GPU 가속 가능)	빠름 (CPU 기반)
RAG 적합성	실험용, 속도 중심	실전용, 유지보수 중심

● 기본 사용 예제 (LangChain + Chroma)

python

chroma_basic_example.py

```
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
```

1. 임베딩 모델 정의

```
embedding = OpenAIEmbeddings(model="text-embedding-3-small")
```

2. 문장 리스트

```
texts = [
    "서울의 봄은 벚꽃이 아름답다.",
    "부산의 해운대는 여름에 관광객이 많다.",
    "제주도는 겨울에도 따뜻하다."
]
```

3. Chroma 벡터 저장소 생성 (자동으로 로컬 DB 생성)

```
chroma_store = Chroma.from_texts(
    texts=texts,
    embedding=embedding,
    collection_name="korea_travel"
)
```

4. 검색

```
query = "겨울에 따뜻한 지역"
results = chroma_store.similarity_search(query, k=2)
```

```
for r in results:
    print(r.page_content)
```

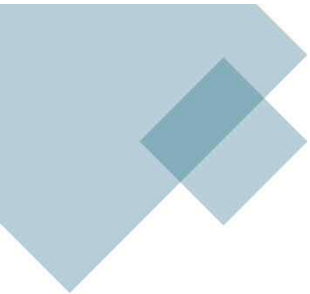

● 디스크에 자동 저장되는 위치

기본적으로 Chroma는 `./chroma/` 폴더를 자동 생성하여 벡터와 메타데이터를 저장한다.

python

```
./chroma/  
├─ index/  
│   ├── data_0.bin  
│   ├── header.bin  
│   └─ ...  
├─ collection/  
│   ├── metadata.sqlite  
│   └─ ...
```

- 별도의 `save_local()` 호출이 없어도 **자동 영구 저장** 된다.
- `persist_directory` 옵션으로 저장 경로를 지정할 수도 있다.



● Chroma의 장점 요약

장점

✓ 자동 저장

설명

벡터가 자동으로 디스크에 영구 저장됨

✓ 메타데이터 필터링

조건(`filter={}`)을 통해 세밀한 검색 가능

✓ 빠른 검색 속도

수십만 벡터 규모에서도 실시간 검색

✓ LangChain 통합

`Chroma.from_texts()`, `Chroma.from_documents()` 로 간단하게 생성 가능

✓ 클라우드 확장

Docker, REST API, Chroma Cloud 모드 지원





● 정리

항목	내용
정의	로컬 또는 서버형 벡터 데이터베이스
주요 메서드	<code>from_texts()</code> , <code>from_documents()</code> , <code>.add_texts()</code> , <code>.delete()</code> , <code>.persist()</code>
저장 방식	디스크 자동 저장 (<code>persist_directory</code>)
필터링 기능	<code>filter={"키":"값"}</code> 조건 검색 지원
적합한 용도	RAG, FAQ, 검색형 에이전트, 문서 관리 서비스 등



● 주요 메서드 정리

동작	메서드	예시 코드	설명
문서 추가	<code>.add_documents(docs)</code>	<code>chroma.add_documents([Document(...), ...])</code>	기존 컬렉션에 새로운 문서 삽입
문서 삭제(조건)	<code>.delete(where={...})</code>	<code>chroma.delete(where={"city": "Seoul"})</code>	metadata 조건 기반 삭제
문서 삭제(전체)	<code>.delete(where={})</code>	<code>chroma.delete(where={})</code>	컬렉션 전체 초기화
문서 삭제(ID)	<code>.delete(ids=[...])</code>	<code>chroma.delete(ids=["id1"])</code>	특정 문서 ID로 삭제
검색	<code>.similarity_search(query, k)</code>	<code>chroma.similarity_search("여행", k=3)</code>	가장 유사한 문서 반환

초기화(reset_collection)

✓ 핵심 개념

항목	설명
메서드명	<code>reset_collection()</code>
역할	Chroma 컬렉션(데이터 전체)을 완전히 초기화
결과	기존의 모든 벡터/문서/메타데이터 삭제
재사용 여부	초기화 후에도 동일한 <code>collection_name</code> 으로 계속 사용 가능
내부 동작	<code>self._client.reset()</code> 또는 새로운 빈 컬렉션 생성 로직 수행

4. PostgreSQL + pgvector



PostgreSQL + pgvector 는 기업형 RAG 시스템이나 LangChain 실무 프로젝트에서 자주 쓰임

● PostgreSQL + pgvector란?

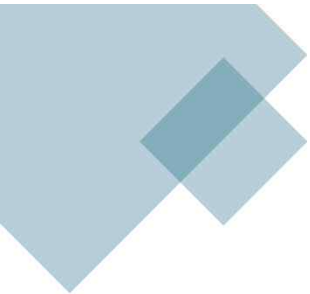
PostgreSQL은 전통적인 관계형 데이터베이스(RDBMS)

pgvector는 그 PostgreSQL에 설치해 사용하는 벡터(Vector) 확장 기능(extension) 입니다.

즉, PostgreSQL을 벡터 데이터베이스처럼 확장시켜서

OpenAI, HuggingFace 등에서 생성한 임베딩(embedding) 벡터를 저장하고 검색할 수 있게 만들어줍니다.





● 핵심 개념 요약

구성 요소

설명

PostgreSQL

오픈소스 SQL 데이터베이스 (관계형 DBMS)

pgvector

PostgreSQL에 설치하는 확장 모듈(extension)

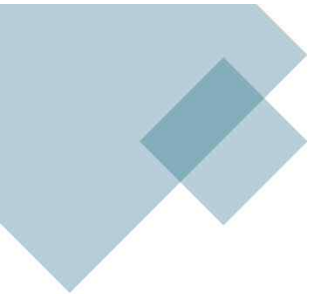
기능

벡터 타입 지원 (`vector`), 벡터 유사도 검색(`cosine` , `L2` , `inner product`)

결과

RDB + Vector Search를 동시에 수행 가능





● 왜 중요한가?

항목

설명

통합 관리

SQL 데이터(메타데이터, 사용자정보 등)와 벡터 임베딩을 한 DB에 저장

엔터프라이즈용

클라우드 관리형 (RDS, AlloyDB, Supabase 등에서 지원)

보안/거버넌스

기존 SQL 권한체계, 백업, 트랜잭션 그대로 유지

RAG 친화적

LangChain, LlamaIndex, Supabase 등과 완전 통합 가능



● 구조 예시

pgsql

PostgreSQL

└─ Table: documents

└─ id (UUID)

└─ content (TEXT)

└─ metadata (JSONB)

└─ embedding (VECTOR) ← pgvector가 제공하는 데이터 타입

● 설치 및 설정

① PostgreSQL 설치 후, 확장 설치

bash

PostgreSQL 접속

psql -U postgres

pgvector 설치

CREATE EXTENSION vector;

또는 Ubuntu에서는 패키지로 설치:

bash

sudo apt install postgresql-15-pgvector

◆ PostgreSQL 설치 방법 (Windows 기준)

1 공식 사이트 이동 <https://www.enterprisedb.com/downloads/postgres-postgresql-downloads>

 <https://www.postgresql.org/download/windows/> ↗

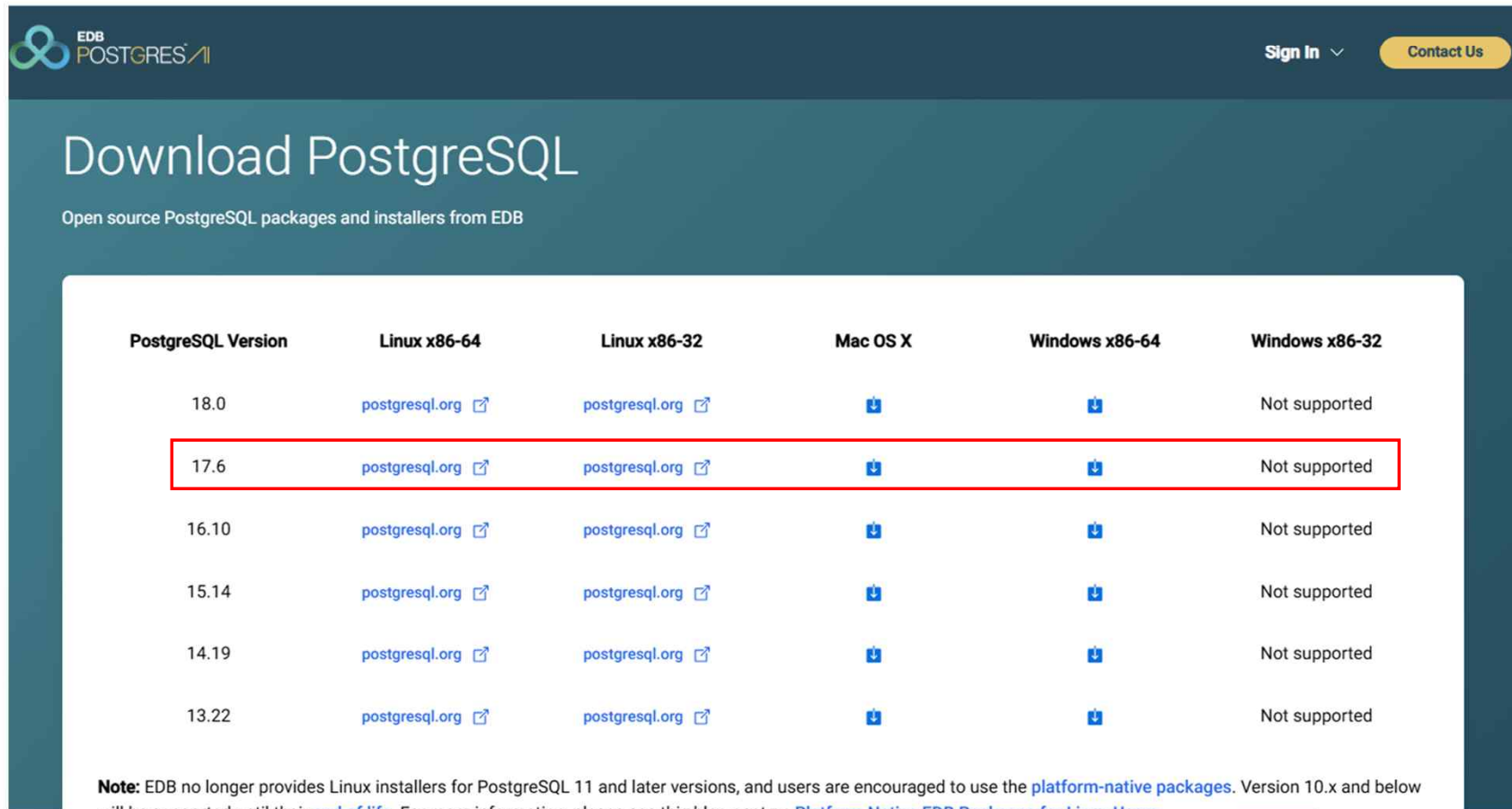
2 "Download the installer" 클릭

→ EnterpriseDB(EDB) 설치 관리자로 이동

3 버전 선택 예시

- Version: 15 (또는 16)
- OS: Windows x86-64
- Download Installer 클릭

postgresql-17.6-1-windows-x64.exe 다운로드 후 설치



EDB
POSTGRES/11

Sign In  [Contact Us](#)

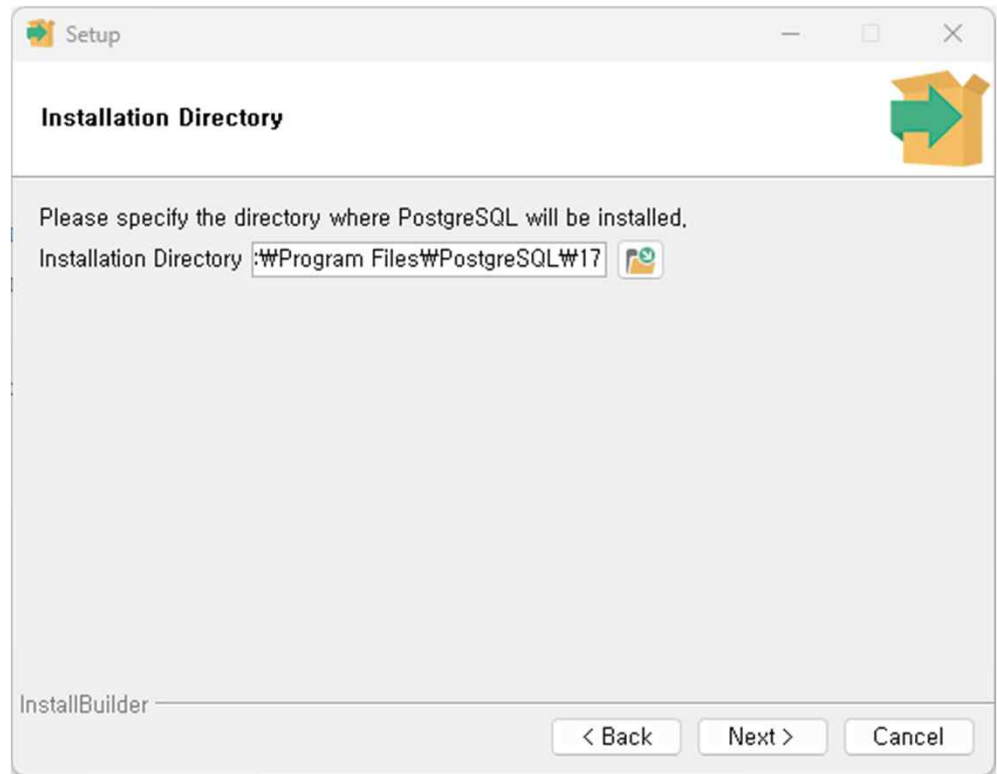
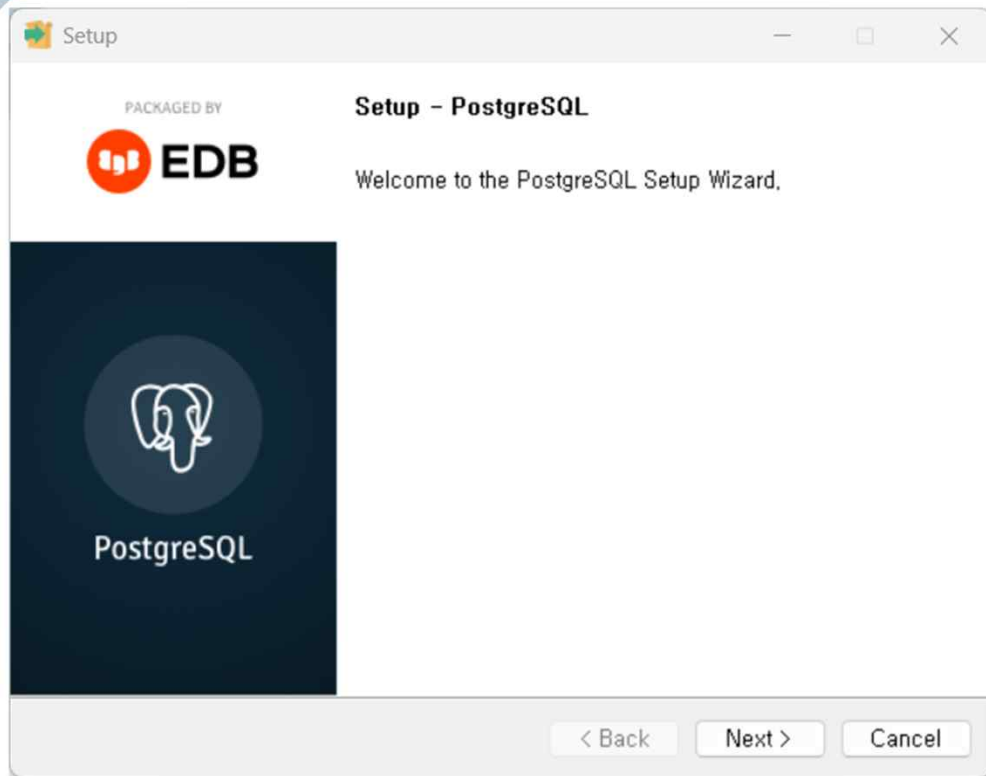
Download PostgreSQL

Open source PostgreSQL packages and installers from EDB

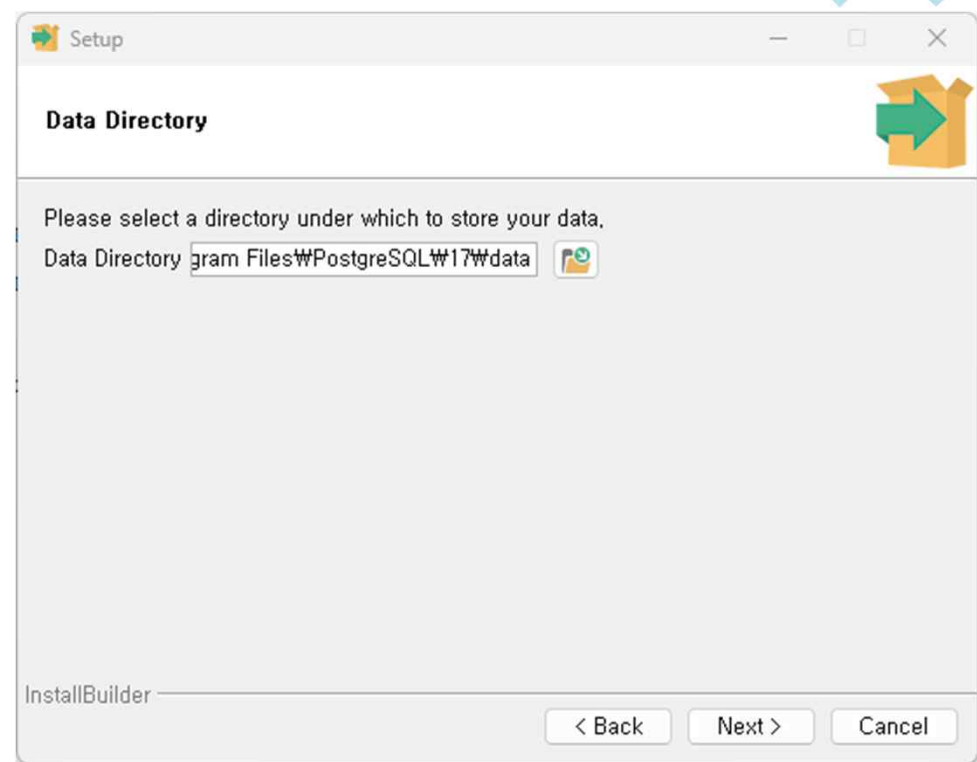
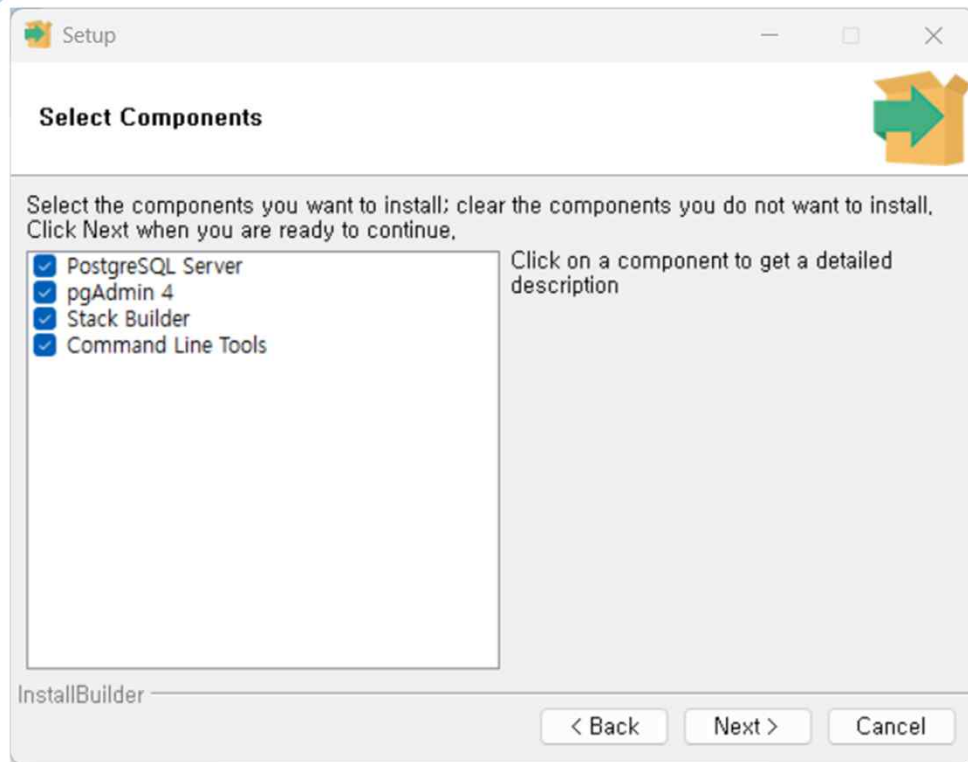
PostgreSQL Version	Linux x86-64	Linux x86-32	Mac OS X	Windows x86-64	Windows x86-32
18.0	postgresql.org 	postgresql.org 			Not supported
17.6	postgresql.org 	postgresql.org 			Not supported
16.10	postgresql.org 	postgresql.org 			Not supported
15.14	postgresql.org 	postgresql.org 			Not supported
14.19	postgresql.org 	postgresql.org 			Not supported
13.22	postgresql.org 	postgresql.org 			Not supported

Note: EDB no longer provides Linux installers for PostgreSQL 11 and later versions, and users are encouraged to use the [platform-native packages](#). Version 10.x and below will be extended until their end of life. For more information, please see this [blog post](#) on Platform Native EDB Packages for Linux Users.

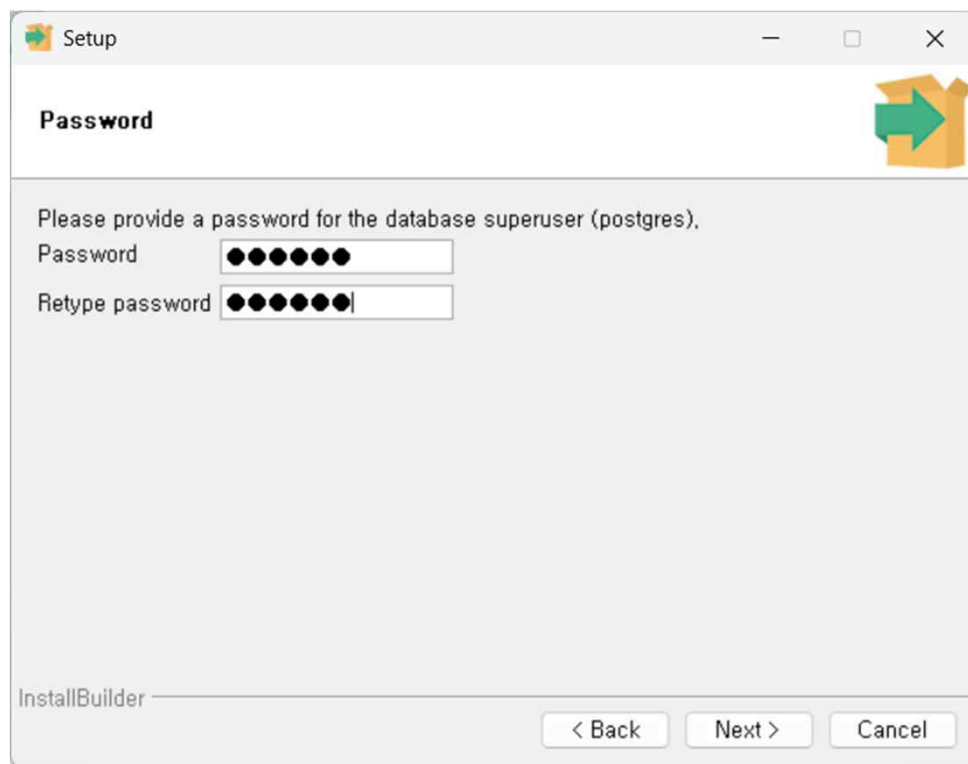
[Next] 버튼 클릭



[Next] 버튼 클릭



비번 입력: 예) 1234



Setup

Password

Please provide a password for the database superuser (postgres).

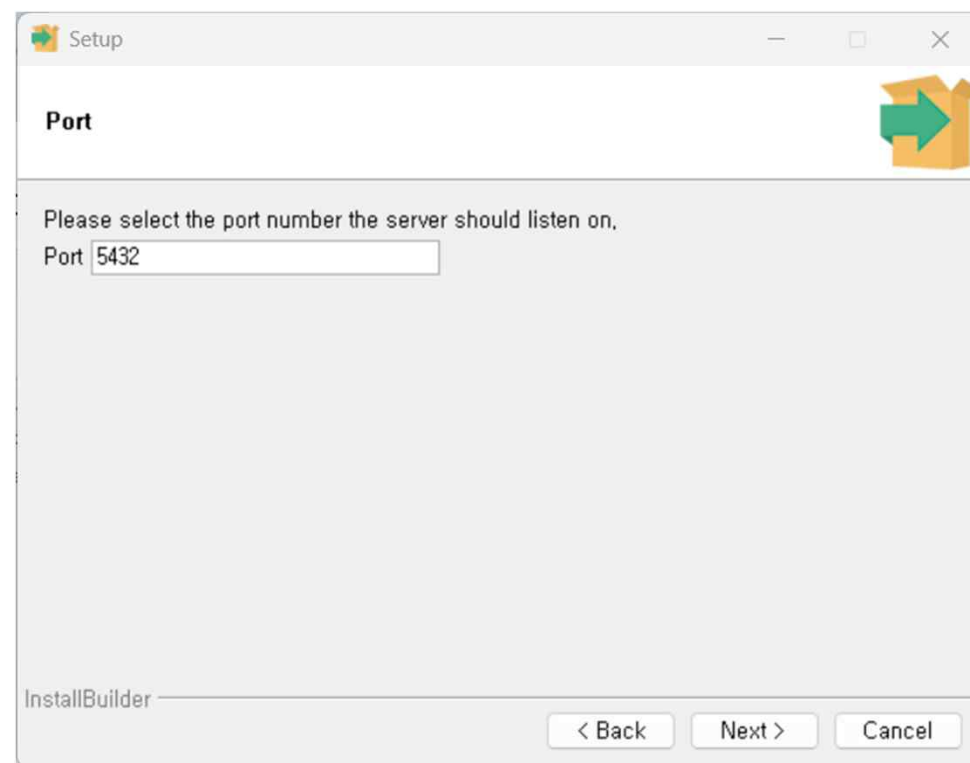
Password

Retype password

InstallBuilder

< Back Next > Cancel

포트번호는 기본값 사용 : 5432



Setup

Port

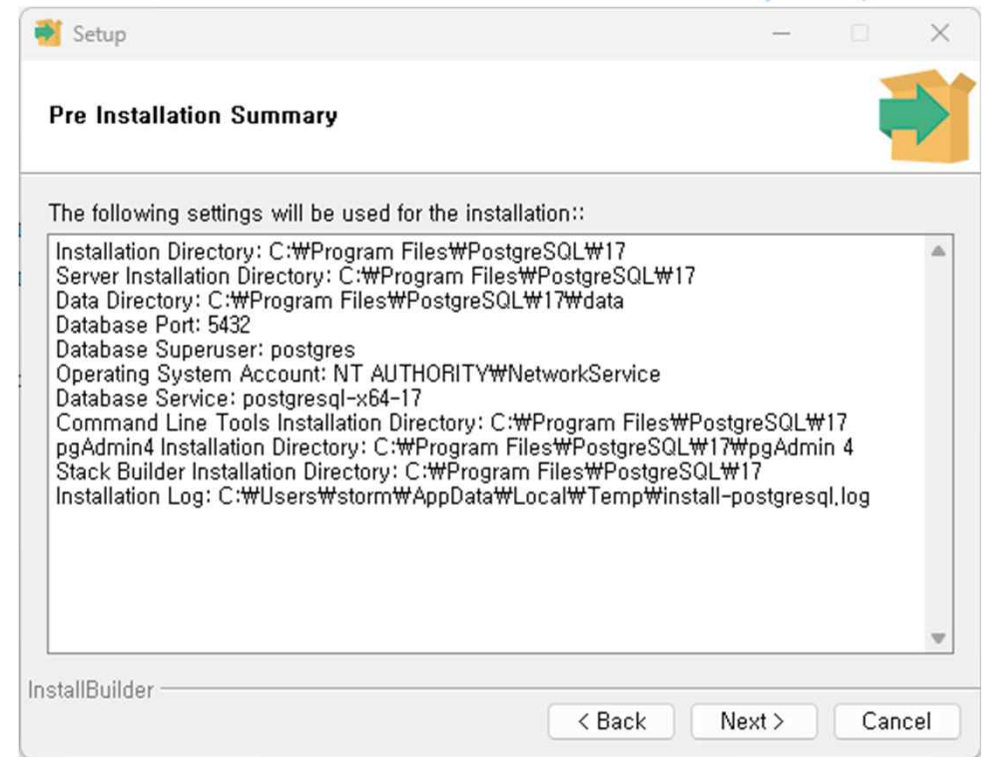
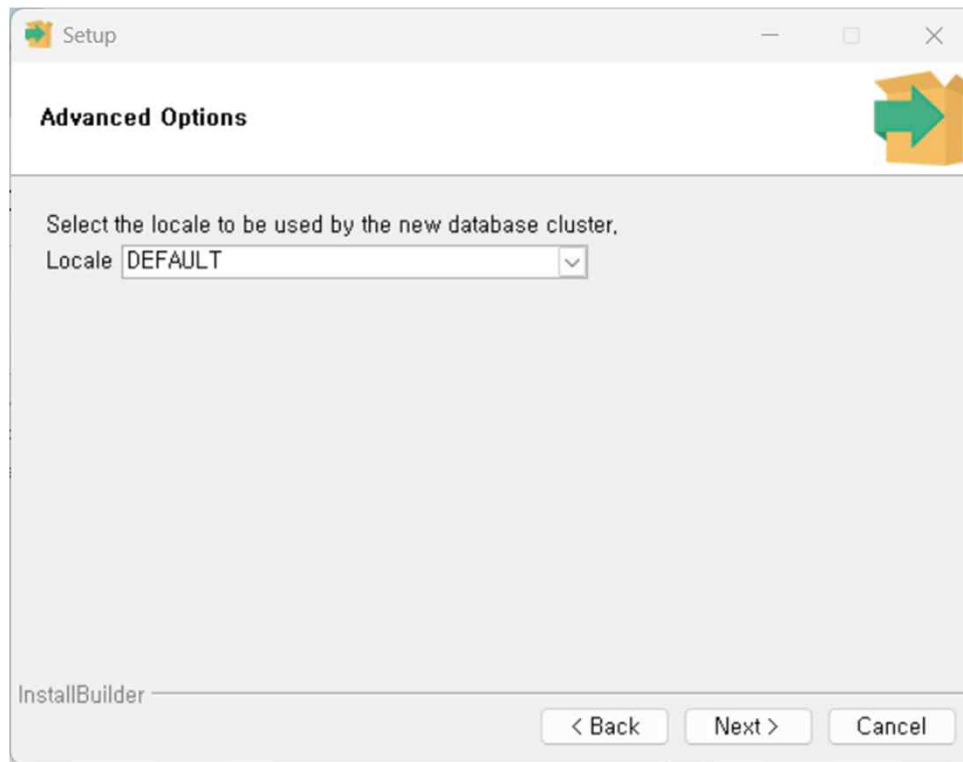
Please select the port number the server should listen on.

Port

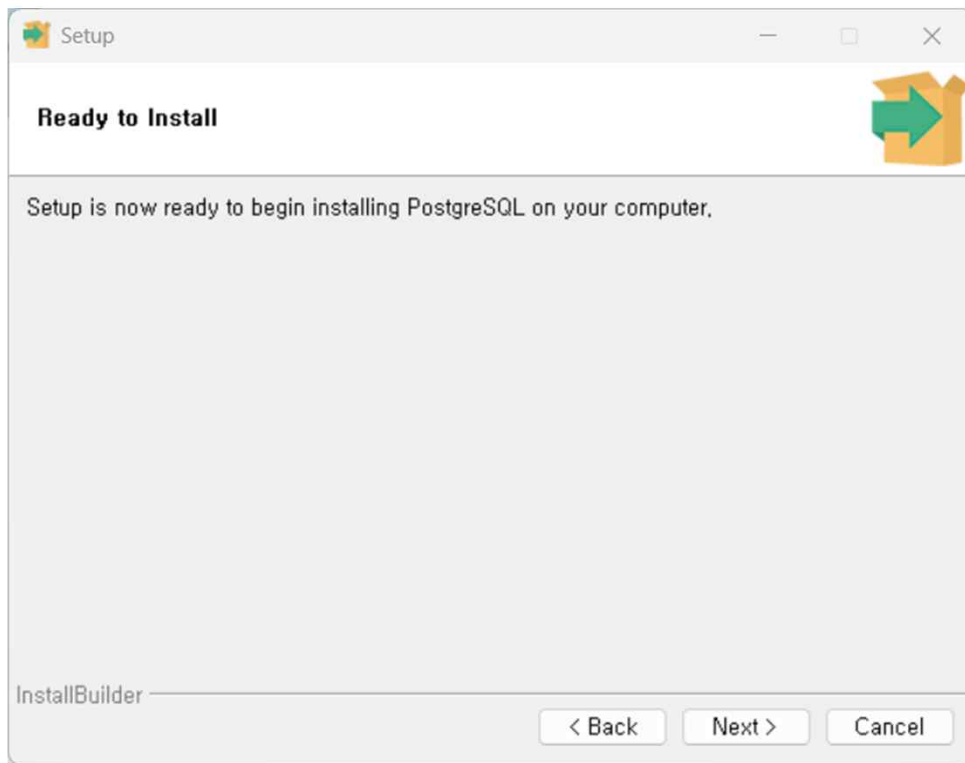
InstallBuilder

< Back Next > Cancel

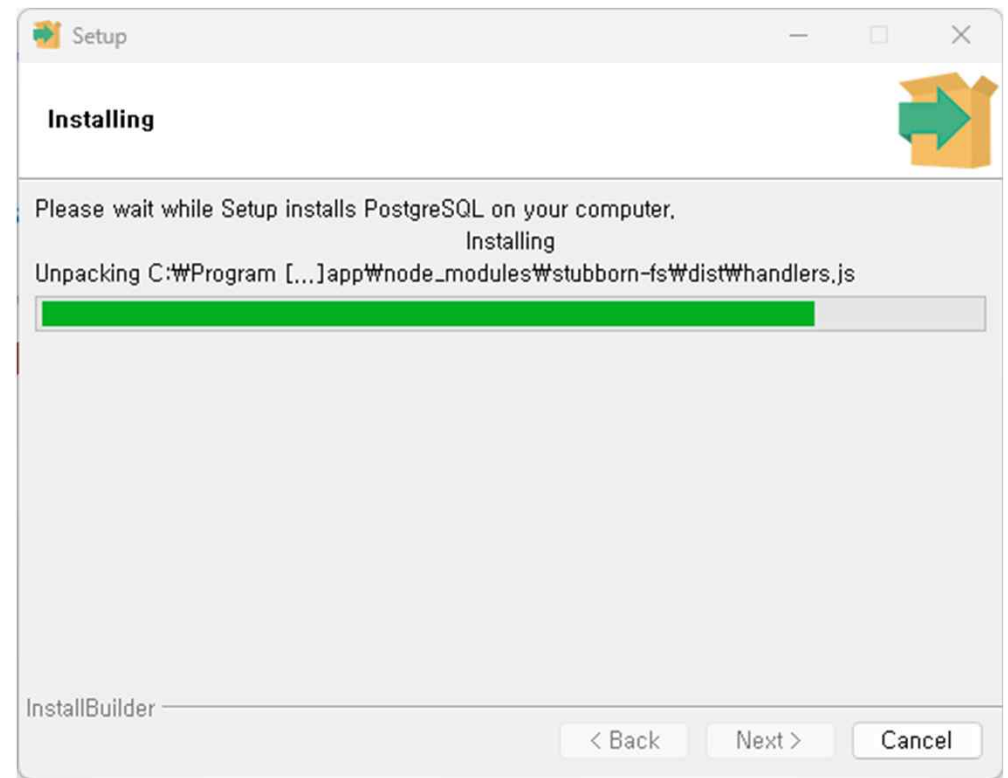
[Next] 버튼 클릭



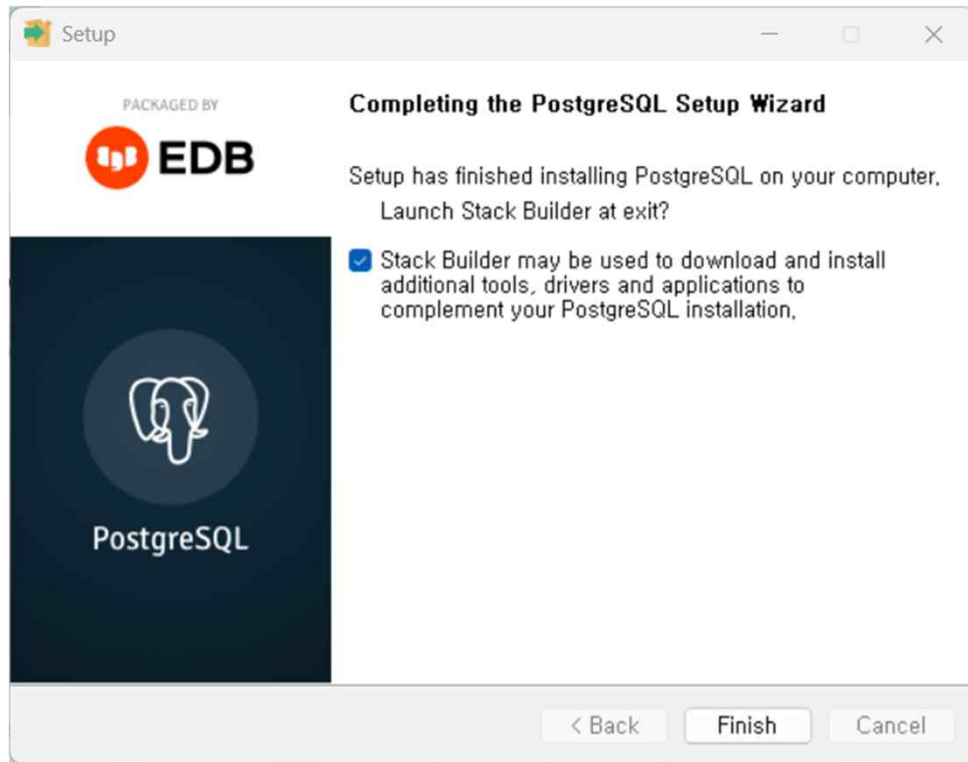
[Next] 버튼 클릭



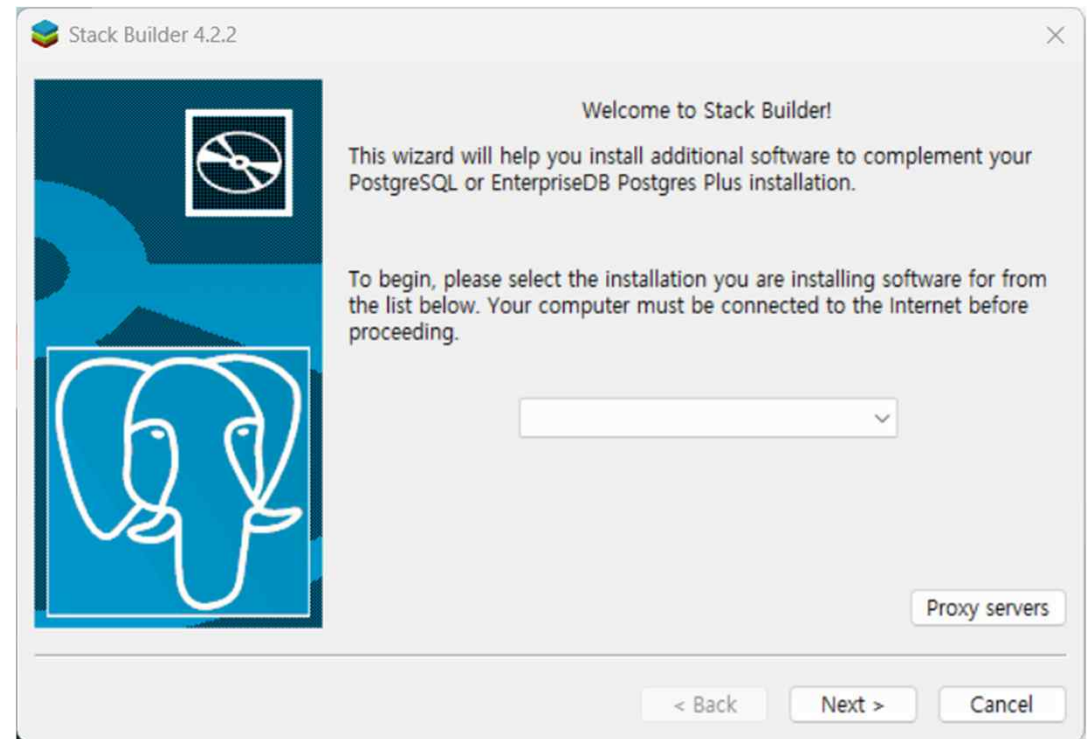
설치 다소 시간 소요됨



[Finish] 버튼 클릭



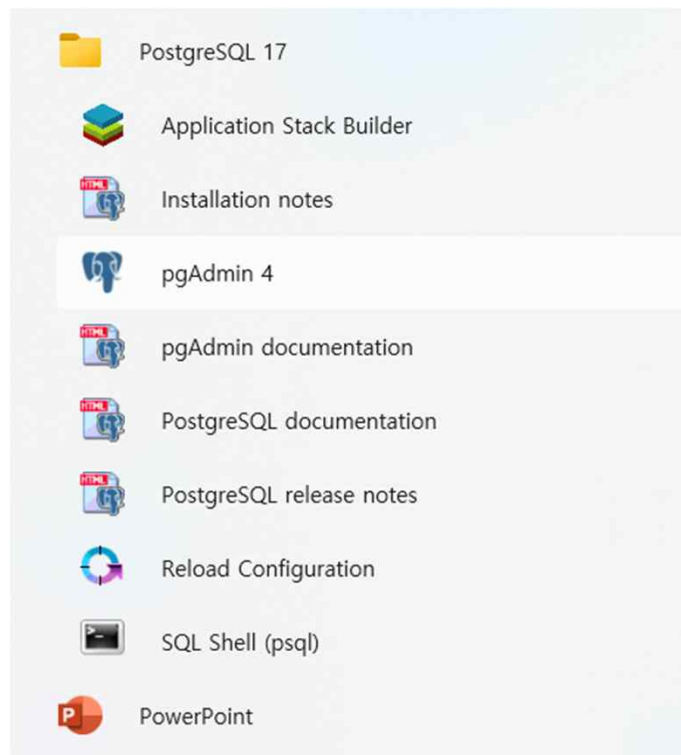
Stack Builder는 추가 모듈 설치용 도구입니다.
단순히 DB만 테스트하려면 Stack Builder는 [Cancel]을
눌러 종료한다



✓ 설치 후 확인 방법

pgAdmin으로 확인

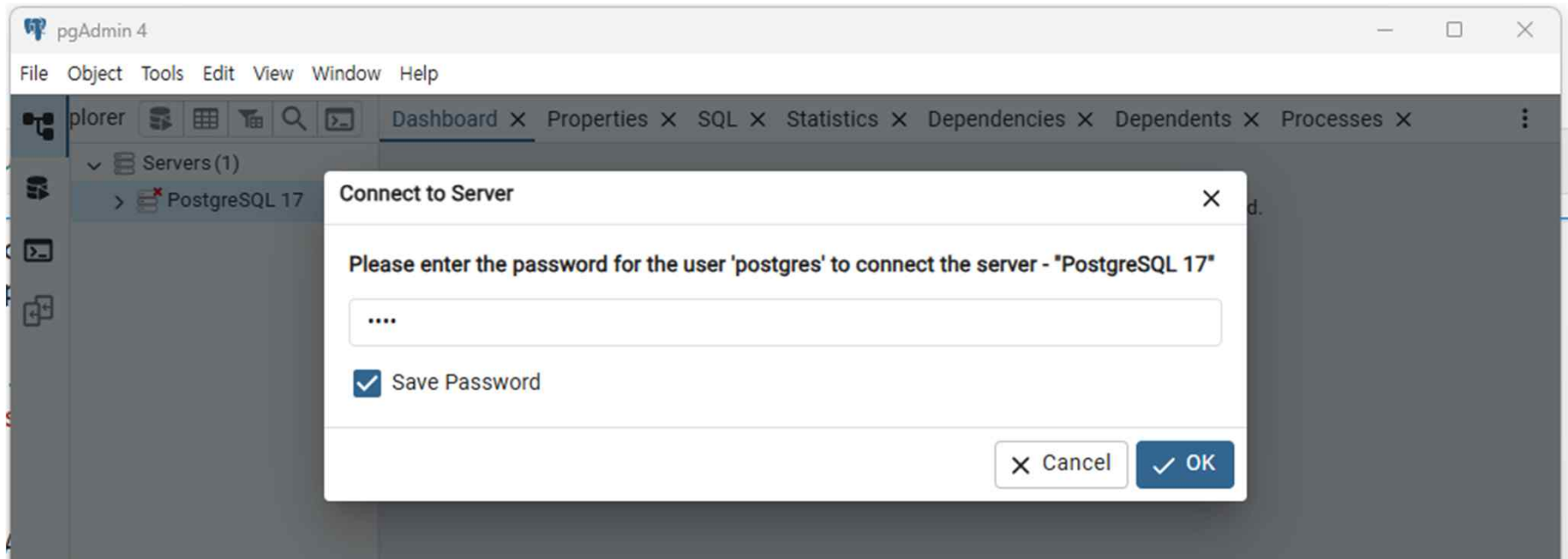
- Windows 시작 메뉴 → “pgAdmin 4” 실행
- 서버 이름 클릭 → 비밀번호(`postgres` 계정 설치 시 지정한 비번) 입력
- 연결 성공하면 정상 작동 중입니다.

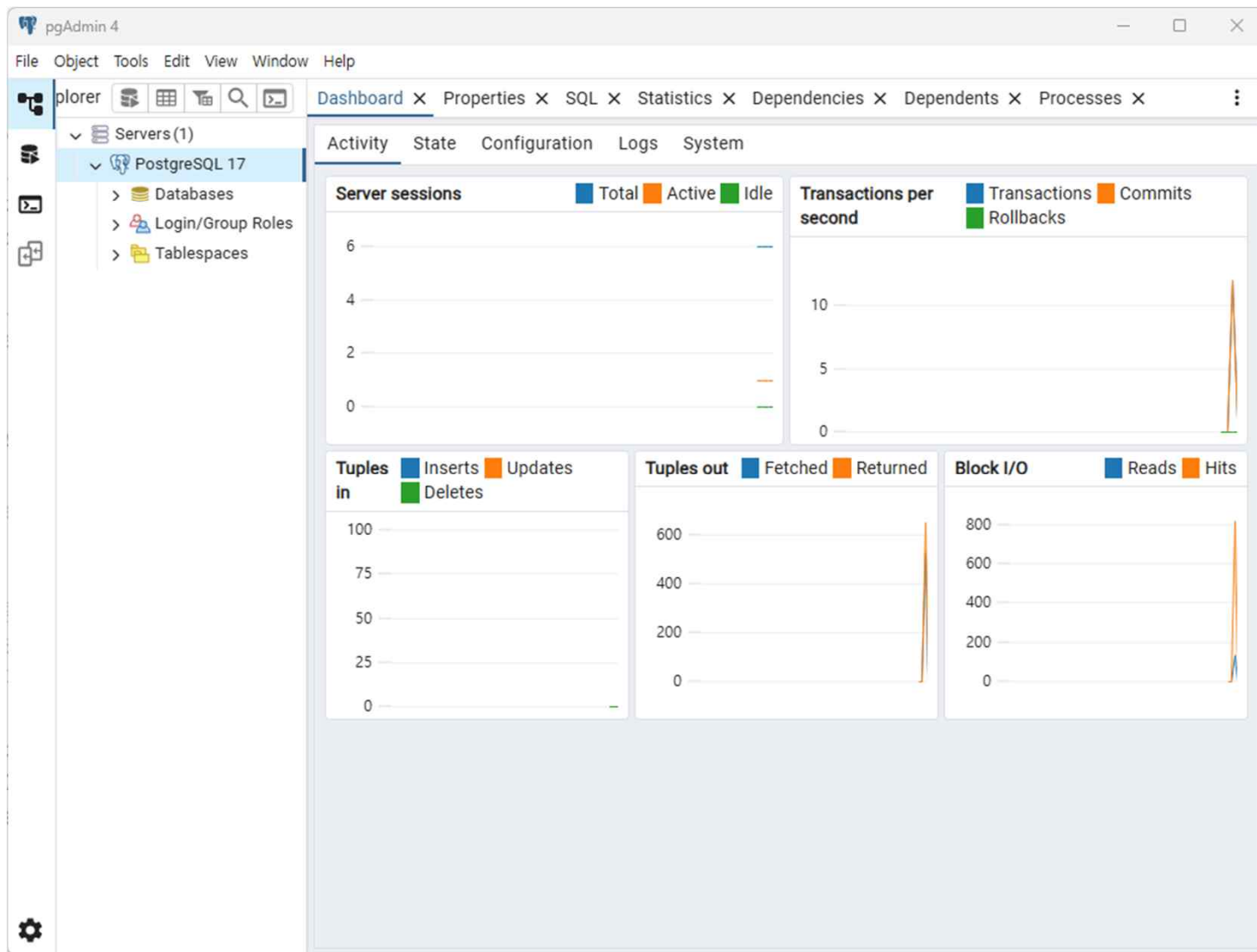


연결될 때까지 기다린다



왼쪽 목록에서 Server를 클릭하고 비번(1234)을 입력하고 [OK] 버튼을 클릭한다





새 데이터베이스 생성하기

윈도우 시작 메뉴에서 **pgAdmin4** 실행 후 왼쪽 트리에서
Databases → 우클릭 → Create → Database 클릭

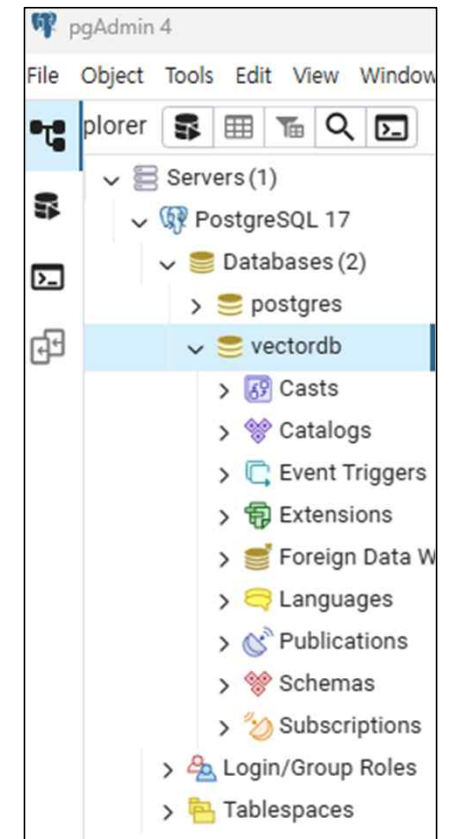
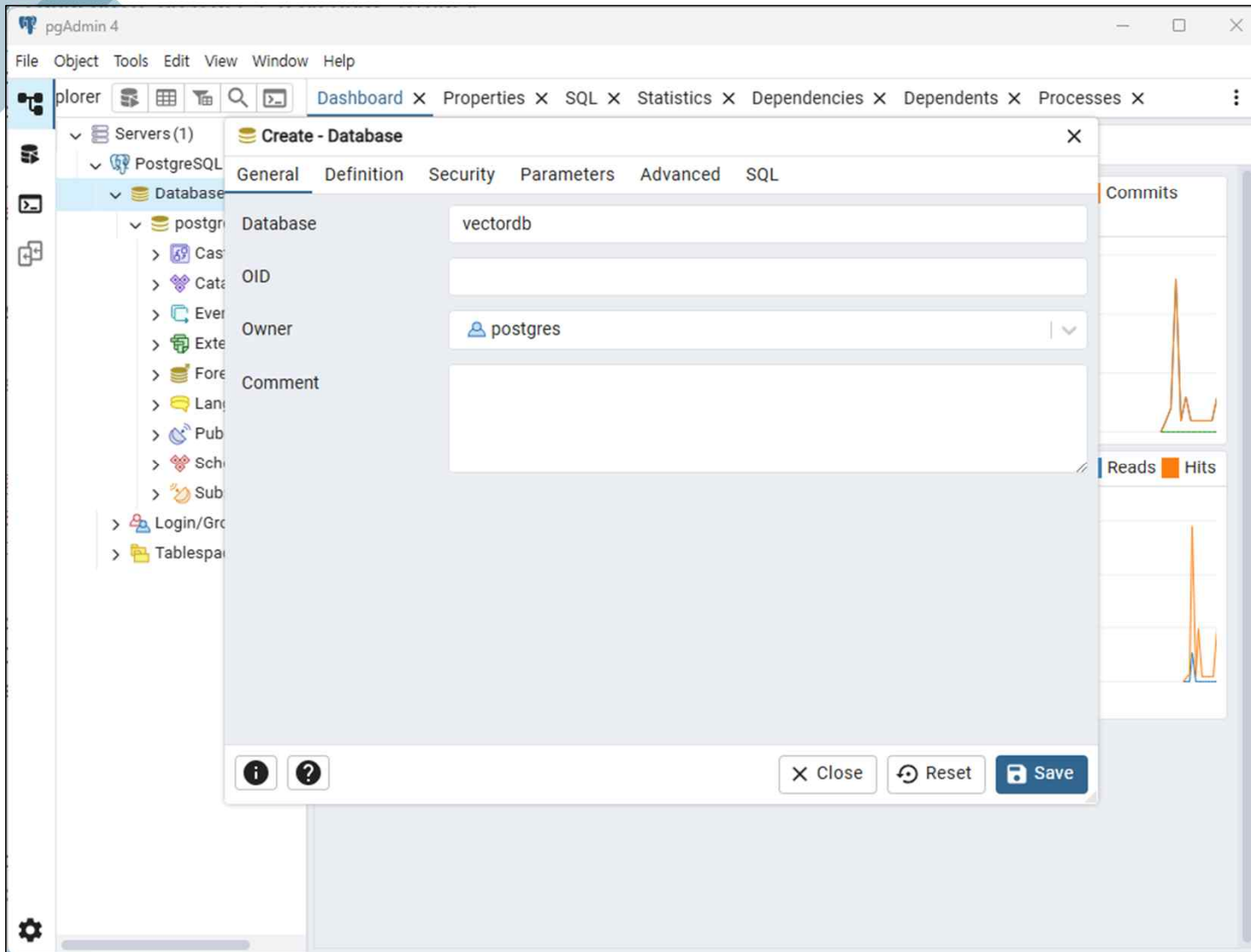
항목

Database

값

vectordb

[Save] 버튼을 클릭하여 저장



Pgvector 설치

* pgvector Windows 빌드 ZIP 다운로드

아래 주소에서 **vector.v0.8.1-pg17.zip** 파일을 다운 받는다

https://github.com/andreiramani/pgvector_pgsql_windows/releases

pgvector v0.8.1 for PostgreSQL 17, Microsoft Windows Latest

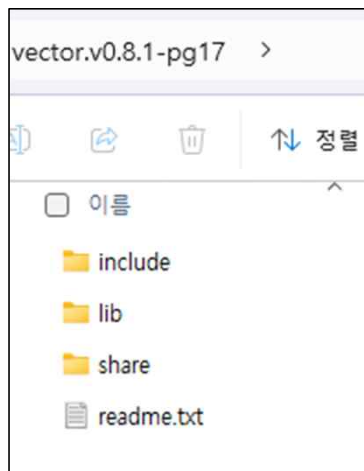
Test in PostgreSQL 17.6 Microsoft Windows OS x64

▼ Assets 3

 vector.v0.8.1-pg17.zip	sha256:0ccf5dafc5a98c290...		161 KB	2 weeks ago
 Source code (zip)				Aug 19
 Source code (tar.gz)				Aug 19



다운로드 후 압축을 풀면 다음 폴더가 보인다



폴더 안의 내용을 PostgreSQL 폴더로 복사한다.
폴더 자체가 아니라 폴더별 파일들을 각각 추가로 복사해
넣는다

폴더 내의 기존 파일들이 삭제 되지 않도록 주의한다

원본 (pgvector 압축 해제 경로)

vector.v0.8.1-pg17\lib*

vector.v0.8.1-pg17\include*

vector.v0.8.1-pg17\share*

복사 대상 (PostgreSQL 설치 경로)

C:\Program Files\PostgreSQL\17\lib\

C:\Program Files\PostgreSQL\17\include\

C:\Program Files\PostgreSQL\17\share\



pgvector 설치를 위해 추가 되는 파일들

C:\Program Files\PostgreSQL\17\include\server\extension\vector\
halfvec.h, sparsevec.h, vector.h

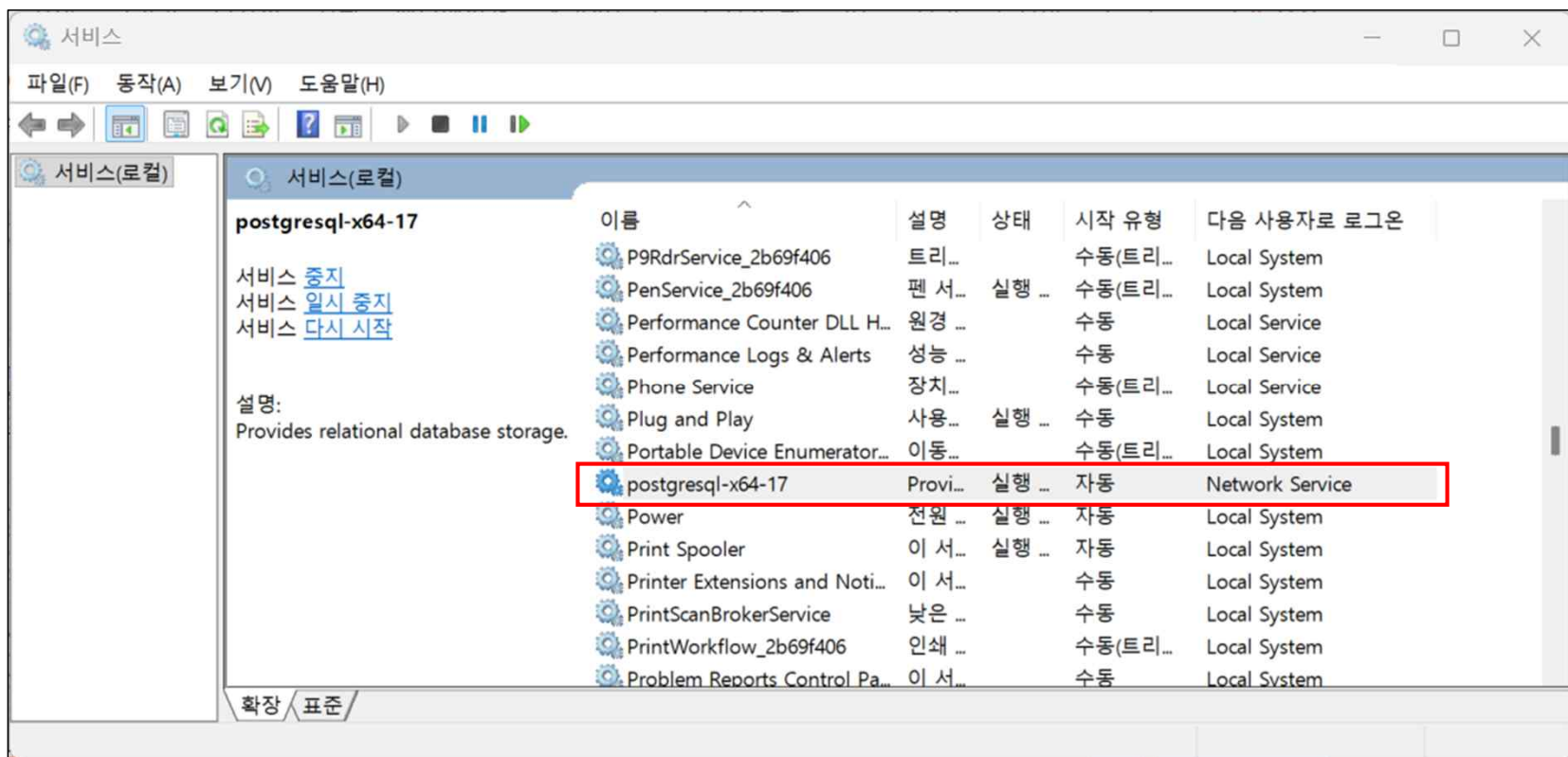
C:\Program Files\PostgreSQL\17\lib\vector.dll

C:\Program Files\PostgreSQL\17\share\extension\
vector.control, vector.sql, vector--0.1.0--0.1.1.sql ... (38개 파일)



PostgreSQL 서비스 재시작

1. Win + R → services.msc 입력
2. postgresql-x64-17 서비스 선택
3. 마우스 오른쪽 클릭 → 다시 시작



pgvector 파이썬 실행 테스트

```
1 from langchain_postgres import PGVector
2 from langchain_openai import OpenAIEmbeddings
3
4 # 1) PostgreSQL 연결 문자열
5 CONNECTION = "postgresql+psycopg2://postgres:1234@localhost:5432/vectordb"
6
7 # 2) 임베딩 모델
8 embedding = OpenAIEmbeddings(model="text-embedding-3-small")
9
10 # 3) Vector Store 생성
11 pgvector_store = PGVector.from_texts(
12     texts=[
13         "서울의 봄은 벚꽃이 아름답다.",
14         "부산의 해운대는 여름에 인기가 많다.",
15         "제주는 겨울에도 따뜻하다."
16     ],
17     embedding=embedding,
18     connection=CONNECTION,
19     collection_name="travel_docs"
20 )
21 print("✅ pgvector 설치 및 연결 성공!")
22 print("✅ 벡터 저장 완료")
23
24 # 검색
25 results = pgvector_store.similarity_search("겨울에 따뜻한 지역", k=2)
26 for r in results:
27     print("🔍", r.page_content)
```

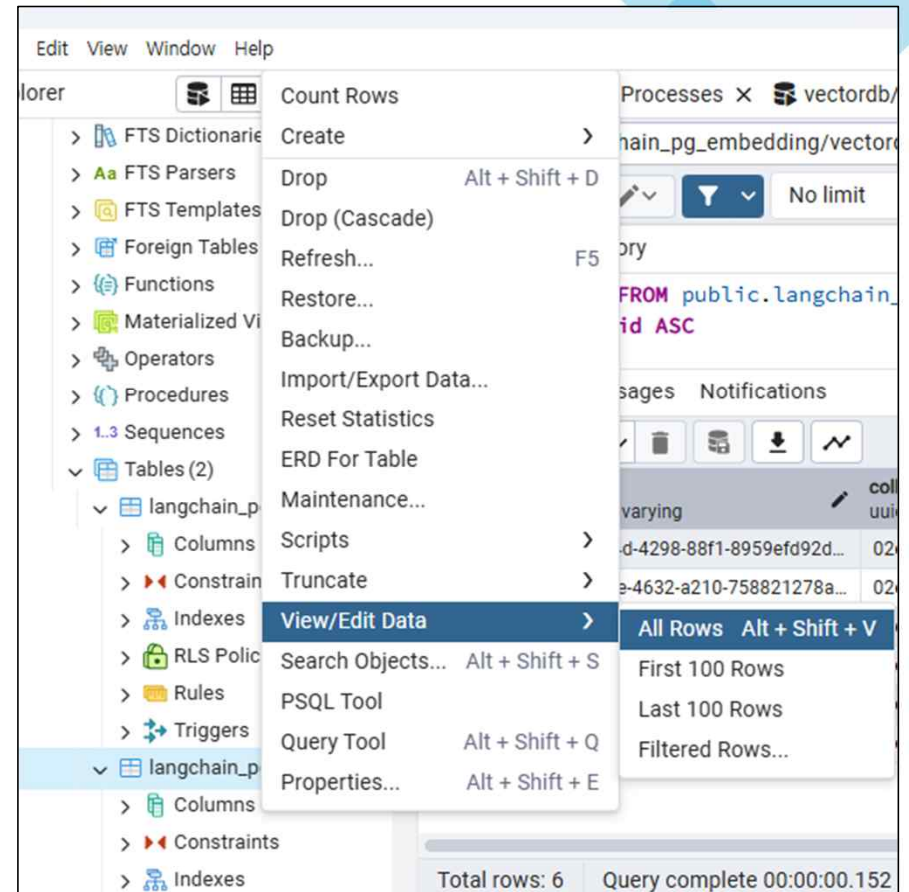
- ✅ pgvector 설치 및 연결 성공!
- ✅ 벡터 저장 완료
- 🔍 제주는 겨울에도 따뜻하다.
- 🔍 제주는 겨울에도 따뜻하다.

PostgreSQL(pgAdmin4)에서 pgvector로 저장된 벡터 데이터(embedding) 를 확인하는 방법

LangChain에서 벡터 저장 시 내부적으로 langchain_pg_embedding 이라는 테이블에 문서, 벡터, 메타데이터가 함께 저장된다.

pgAdmin4 실행 → 왼쪽 트리 열기
Servers → PostgreSQL 17 → Databases → vectordb → Schemas → public → Tables 에서

langchain_pg_embedding 테이블을 선택하고 마우스 우측버튼을 클릭한다음
View/Edit Data → All Rows 클릭



pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- FTS Dictionaries
- FTS Parsers
- FTS Templates
- Foreign Tables
- Functions
- Materialized Views
- Operators
- Procedures
- Sequences
- Tables (2)
 - langchain_pg_collection
 - Columns
 - Constraints
 - Indexes
 - RLS Policies
 - Rules
 - Triggers
 - langchain_pg_embedding
 - Columns
 - Constraints
 - Indexes

public.langchain_pg_embedding/vectoradb/postgres@PostgreSQL 17

Query

```
1 SELECT * FROM public.langchain_pg_embedding
2 ORDER BY id ASC
```

Data Output

Showing rows: 1 to 6 Page No: 1 of 1

	id [PK] character varying	collection_id uuid	embedding vector
1	2e9c3304-254d-4298-88f1-8959efd92d...	02e883da-9eb0-4579-8e35-00db4ae06df7	[0.037546724,-0.015654465,0.03992588,0.062937744,0.036047254,-0.01965
2	3eed39a6-acfe-4632-a210-758821278a...	02e883da-9eb0-4579-8e35-00db4ae06df7	[0.03458462,-0.027845139,-0.014545332,0.005737088,0.016916946,-0.0375
3	671a7b2e-28db-4deb-a97c-410098dae...	02e883da-9eb0-4579-8e35-00db4ae06df7	[0.03450465,0.0055050147,-0.052415606,0.025421372,0.05249425,-0.02032
4	b5dee64d-bbe3-4ccd-8871-0f77f3bf75...	02e883da-9eb0-4579-8e35-00db4ae06df7	[0.03450465,0.0055050147,-0.052415606,0.025421372,0.05249425,-0.02032
5	ea8a0f4e-e12c-4ceb-a5b6-ae05dcc4d...	02e883da-9eb0-4579-8e35-00db4ae06df7	[0.037644524,-0.015653566,0.039883602,0.06293413,0.036065172,-0.01964
6	fe86dea3-9df5-4c2e-973a-de0546606fc7	02e883da-9eb0-4579-8e35-00db4ae06df7	[0.03458462,-0.027845139,-0.014545332,0.005737088,0.016916946,-0.0375

Total rows: 6 Query complete 00:00:00.152 CRLF Ln 1, Col 1

SQL 창에서 데이터 조회

pgAdmin4의 Query Tool 열고 아래 SQL 실행:

```
SELECT id, document, embedding, cmetadataFROM  
langchain_pg_embeddingLIMIT 5;
```

The screenshot shows the pgAdmin4 Query Tool interface. The top bar includes tabs for Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, Processes, and a connection named 'vectordb/postgres...'. The connection dropdown shows 'vectordb/postgres@PostgreSQL 17'. Below the toolbar, the 'Query' tab is active, displaying the following SQL query:

```
1 SELECT id, document, embedding, cmetadata
2 FROM langchain_pg_embedding
3 LIMIT 5;
```

The 'Data Output' tab is selected, showing the results of the query in a table. The table has three columns: 'id' (character varying), 'document' (character varying), and 'embedding' (vector). The results are as follows:

	id [PK] character varying	document character varying	embedding vector
1	fe86dea3-9df5-4c2e-973a-de0546606fc7	서울의 봄은 벚꽃이 아름답다.	[0.03458462,-0.027845139,-0.014545332,0.005737088,0.016916946,-0.0...
2	2e9c3304-254d-4298-88f1-8959efd92d7f	부산의 해운대는 여름에 인기가 많다.	[0.037546724,-0.015654465,0.03992588,0.062937744,0.036047254,-0.0...
3	671a7b2e-28db-4deb-a97c-410098dae7...	제주는 겨울에도 따뜻하다.	[0.03450465,0.0055050147,-0.052415606,0.025421372,0.05249425,-0.0...
4	3eed39a6-acfe-4632-a210-758821278a...	서울의 봄은 벚꽃이 아름답다.	[0.03458462,-0.027845139,-0.014545332,0.005737088,0.016916946,-0.0...
5	ea8a0f4e-e12c-4ceb-a5b6-ae05dcc4d81	부산의 해운대는 여름에 인기가 많다.	[0.037644524,-0.015653566,0.039883602,0.06293413,0.036065172,-0.0...

SQLAlchemy

✓ 1 SQLAlchemy란?

SQLAlchemy는

Python에서 데이터베이스(SQL)를 쉽게 다룰 수 있게 해주는 ORM(객체 관계 매핑) 및 SQL 도구입니다.

✓ 2 SQLAlchemy의 핵심 구성

구성요소	역할	설명
Engine	DB 연결 객체	실제 DB와의 연결(세션 관리, 커넥션 풀링)
Connection	SQL 실행 객체	쿼리 실행, 결과 반환
Session	ORM 모드에서 트랜잭션 관리	<code>commit</code> , <code>rollback</code> 등 처리
Model	Python 클래스로 정의한 테이블	ORM 방식으로 SQL 없이 조작 가능
<code>text()</code>	순수 SQL 문을 실행할 때 사용	직접 SQL을 문자열로 전달

✓ 2 SQLAlchemy의 핵심 구성

구성요소	역할	설명
Engine	DB 연결 객체	실제 DB와의 연결(세션 관리, 커넥션 풀링)
Connection	SQL 실행 객체	쿼리 실행, 결과 반환
Session	ORM 모드에서 트랜잭션 관리	<code>commit</code> , <code>rollback</code> 등 처리
Model	Python 클래스로 정의한 테이블	ORM 방식으로 SQL 없이 조작 가능
<code>text()</code>	순수 SQL 문을 실행할 때 사용	직접 SQL을 문자열로 전달

✓ 3 SQLAlchemy 두 가지 사용 방식

방식	설명	예시
Core	SQL 직접 작성 (Raw SQL)	<code>conn.execute(text("SELECT * FROM user"))</code>
ORM	Python 클래스 기반 조작	<code>session.query(User).filter_by(id=1)</code>

✓ 4 기본 구조 (Core 방식)

python

```
from sqlalchemy import create_engine, text

# ④ PostgreSQL 연결 문자열
engine = create_engine("postgresql+psycopg2://postgres:1234@localhost:5432/vectordb")

# ④ 연결 후 SQL 실행
with engine.connect() as conn:
    result = conn.execute(text("SELECT * FROM langchain_pg_embedding LIMIT 3;"))

    for row in result:
        print(row)
```

- ◆ `create_engine()` → DB와 연결하는 핵심 객체
- ◆ `text()` → SQL 구문을 감싸 실행 가능
- ◆ `result` 는 iterable → 한 줄씩 결과 접근 가능

✓ 5 ORM 방식 예시 (테이블을 클래스로 표현)

python

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base, sessionmaker

Base = declarative_base()

# ㉠ 테이블을 Python 클래스로 정의
class User(Base):
    __tablename__ = "user"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    email = Column(String)

# ㉡ DB 연결 및 세션 생성
engine = create_engine("sqlite:///example.db") # PostgreSQL도 가능
Base.metadata.create_all(engine)
Session = sessionmaker(bind=engine)
session = Session()

# ㉢ 데이터 추가
new_user = User(name="홍길동", email="hong@example.com")
session.add(new_user)
session.commit()

# ㉣ 데이터 조회
for user in session.query(User):
    print(user.name, user.email)
```

✅ 6 LangChain과 SQLAlchemy의 관계

LangChain의 **PGVector** 나 **SQL Database Toolkit** 내부에서도 SQLAlchemy를 사용합니다.

예를 들어 LangChain의 `PGVector` 초기화 시:

```
python

from langchain_postgres import PGVector
from langchain_openai import OpenAIEmbeddings

pgvector_store = PGVector(
    connection="postgresql+psycopg2://postgres:1234@localhost:5432/vectordb",
    embedding_function=OpenAIEmbeddings(model="text-embedding-3-small"),
    collection_name="travel_docs"
)
```

내부적으로는

```
python

engine = create_engine(connection_string)
```

이 실행되어 SQLAlchemy Engine을 사용하게 됩니다.

즉, LangChain이 자동으로 SQLAlchemy를 통해 PostgreSQL과 통신합니다.

✓ 7 주요 SQLAlchemy 함수 정리

함수/메서드

설명

`create_engine()`

DB 연결 생성

`engine.connect()`

연결 객체 생성

`text()`

SQL 문자열 감싸기

`conn.execute()`

SQL 실행

`result.fetchall()`

결과 전체 조회

`Session()`

ORM용 세션 생성

`Base.metadata.create_all()`

테이블 자동 생성

`Column()`, `Integer`, `String`

ORM 필드 정의용



8

다양한 DB 연결 문자열 예시

DB	연결 문자열 예시
PostgreSQL	"postgresql+psycopg2://user:pass@localhost:5432/dbname"
MySQL	"mysql+pymysql://user:pass@localhost:3306/dbname"
SQLite	"sqlite:///example.db"
MSSQL	"mssql+pyodbc://user:pass@server/dbname? driver=ODBC+Driver+17+for+SQL+Server"

5. Pinecone



● Pinecone 이란?

Pinecone은 완전 관리형(Managed) 클라우드 벡터 데이터베이스로, 임베딩(embedding)된 데이터를 저장하고, 빠르게 유사도 검색(similarity search)을 수행하기 위해 설계된 서비스입니다.

즉, FAISS나 Chroma는 로컬(Local) 벡터DB,
Pinecone은 클라우드 기반(Vector DBaaS) 이라고 보면 됩니다.

● 개발사 및 특징

항목	내용
개발사	Pinecone Systems Inc. (미국, 2021년 출시)
홈페이지	https://www.pinecone.io ↗
형태	클라우드형 Vector Database (SaaS)
관리 방식	완전관리형 — 인프라, 확장성, 백업, 인덱싱 자동
주요 사용처	OpenAI, LangChain, LlamaIndex, Cohere, Anthropic 등
언어 SDK	Python, Node.js, Go 등

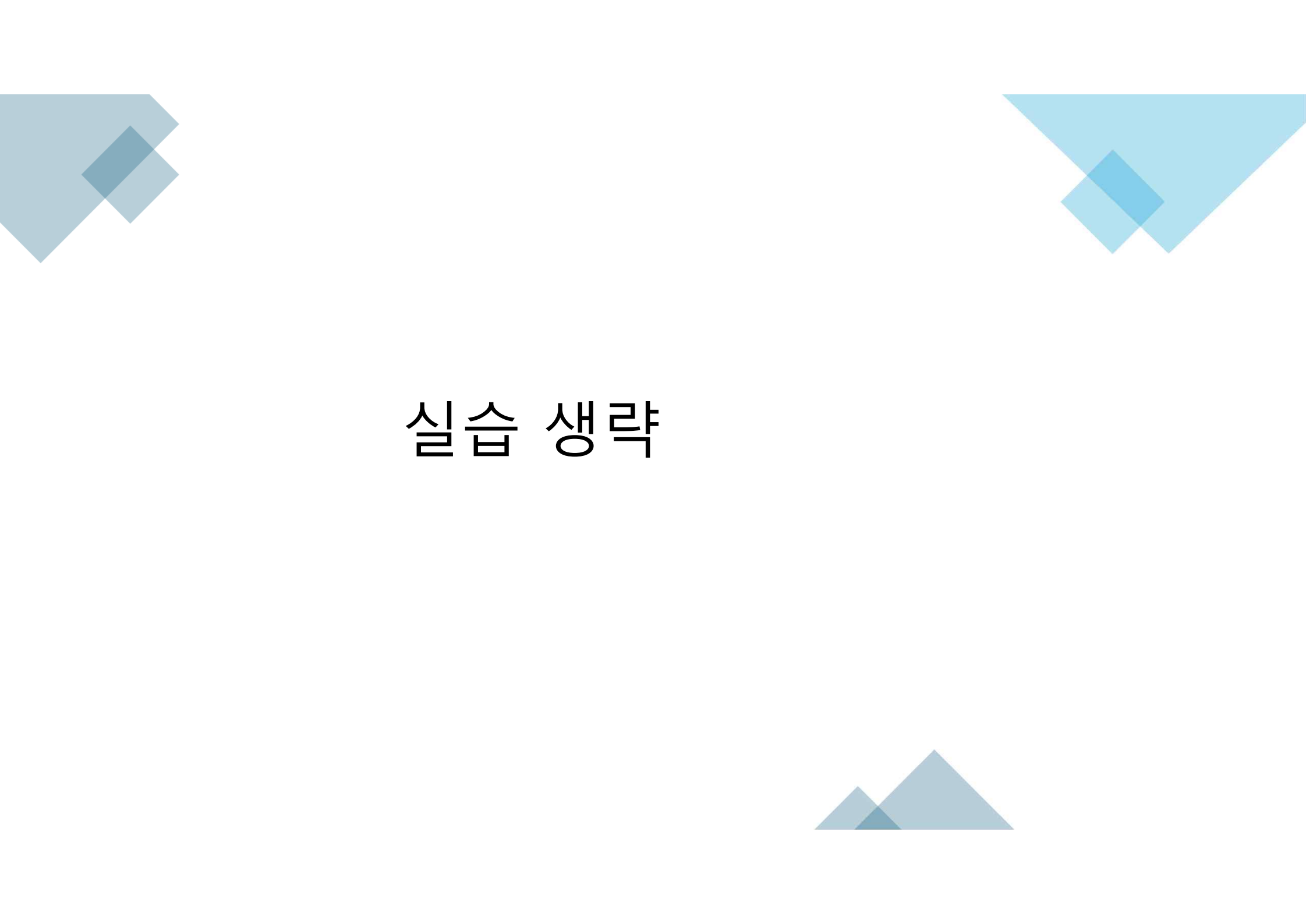
Pinecone은 클라우드형 벡터 데이터베이스이므로
모든 요청은 API를 통해 이루어지고,
따라서 **유료 API Key 없이는 사용이 불가능**합니다.

● 주요 기능

기능	설명
1. 벡터 저장 및 검색	텍스트, 이미지 등에서 생성한 임베딩 벡터를 고속 검색
2. 스케일 아웃(Scale-out)	대규모 데이터(수억 개 벡터)도 분산처리로 자동 확장
3. 필터링 검색	metadata 조건(<code>filter={}</code>)을 통한 세부 검색 지원
4. 자동 인덱싱	벡터 추가 시 실시간 인덱싱 처리
5. 클라우드 관리형	서버 유지보수, 스토리지 확장, 복제 자동 처리
6. RAG 통합 지원	LangChain, LlamaIndex, OpenAI 등과 완전 호환

● FAISS / Chroma / Pinecone 비교

비교 항목	FAISS	Chroma	Pinecone
설치 위치	로컬	로컬 (혹은 Docker)	클라우드 (SaaS)
운영 주체	직접 관리	직접 관리	Pinecone 서버에서 자동 관리
저장 방식	메모리 / 파일	디스크(Local DB)	원격 API 기반 저장
속도	매우 빠름 (GPU 지원)	빠름 (CPU 기반)	빠름 (클라우드 최적화)
확장성	수동	제한적	무제한 (자동 확장)
비용	무료	무료	무료 플랜 + 유료 확장
통합 지원	LangChain	LangChain	LangChain, LlamaIndex, OpenAI 등 완벽 지원
사용 예시	연구, 실험	개인 프로젝트	실무 RAG 서비스, 프로덕션 환경

The slide features four decorative geometric shapes in the corners. The top-left and bottom-right corners contain overlapping triangles and squares in various shades of blue. The top-right and bottom-left corners contain overlapping triangles and squares in various shades of light blue. The central text is in a clean, black, sans-serif font.

실습 생략



감사합니다